03 — DDD (Domain-Driven Design)

Requisitos: **6** (Verificar situação do cliente), **7** (Remover cliente), **10** (Verificar situação da conta corrente) e **11** (Remover conta corrente).

Objetivo: **não** montar uma arquitetura DDD completa, e sim mostrar que cada regra está na classe que naturalmente a possui. Conceitos de DDD usados, sem nada além disso:

- **Linguagem ubíqua**: nomes do negócio no código (`ativar()`, `SituacaoCliente.INATIVO`, `possuiSaldoSuficiente`).
- **Entidade**: `Cliente` e `ContaCorrente` têm identidade (`id`) e comportamento próprio. Após a auditoria, o `id` passou a ser `final` (sem `setId`): a identidade de uma entidade não muda.
- **Value Object** simples: `SituacaoCliente` e `SituacaoConta` (enums imutáveis, sem identidade).
- **Repositório**: `GerenciadoraClientes`/`GerenciadoraContas` (já existiam) cuidam da coleção de entidades.
- **Serviço/fachada**: `SistemaBancario` coordena operações que envolvem repositório e entidades.

Testes antes da modelagem (`evidencias/28-ddd-testes-antes-da-modelagem.txt`): os testes de `test/java/ddd/` foram escritos primeiro e não compilavam (`cannot find symbol: class SituacaoCliente`, `method removerCliente(int)` ...). Depois da modelagem, a suíte inteira passou (`evidencias/29-ddd-apos-modelagem-suite-completa.txt`): `Tests run: 39, Failures: 0`, e o `TesteUnitarioManual` continuou 25/25.

---

6 — Verificar situação do cliente

Conceito de domínio envolvido

`Cliente` / `SituacaoCliente`

Regra de negócio

Um cliente está em uma de duas situações: **ATIVO** ou **INATIVO**. Ele muda de situação pelas ações de domínio *ativar* e *desativar*. Verificar a situação de um cliente **inexistente** não é o mesmo que ele estar inativo.

Classes responsáveis anteriormente

- `Cliente`: campo `boolean ativo`, `isAtivo()`, `setAtivo(boolean)`.
- `SistemaBancario.ativarCliente/desativarCliente`: chamavam `cliente.setAtivo(true/false)`.
- `RepositorioBase.estaAtivo(id)`: `item != null && item.isAtivo()`.

Problema identificado

1. A situação era representada apenas por um `boolean`, o que reduz a expressividade do domínio. `setAtivo(false)` é uma instrução técnica, não uma ação do negócio.
2. `estaAtivo(id)` devolve `false` tanto para cliente **inativo** quanto para cliente **inexistente**, misturando dois fatos diferentes.

Modelagem proposta

- Novo enum `negocio.SituacaoCliente { ATIVO, INATIVO }`, com `getDescricao()` usado no `toString()`.
- `Cliente` guarda `SituacaoCliente situacao` e ganha `getSituacao()`, `ativar()`, `desativar()` e `estaAtivo()`.
- `isAtivo()` (exigido pela interface `EntidadeBancaria`) passou a delegar para `estaAtivo()`.
- `setAtivo(boolean)` e o construtor com `boolean ativo` foram **mantidos por compatibilidade** (`CargaInicialFactory`, `TesteUnitarioManual`); internamente chamam `ativar()`/`desativar()`.
- `SistemaBancario.ativarCliente/desativarCliente` passaram a chamar `cliente.ativar()`/`cliente.desativar()`.
- Nova operação `SistemaBancario.verificarSituacaoCliente(id)`, que devolve `Optional<SituacaoCliente>`. Optional **vazio** = cliente não existe.

```
public void ativar()        { this.situacao = SituacaoCliente.ATIVO; }
public void desativar()     { this.situacao = SituacaoCliente.INATIVO; }
public boolean estaAtivo()  { return situacao == SituacaoCliente.ATIVO; }
```

Justificativa

A situação não é tratada como um boolean genérico: ela é um conceito explícito do domínio bancário. A mudança de situação é feita por ações com nome de negócio (`ativar`, `desativar`), dentro da própria entidade, que protege seu estado. O `Optional` torna explícita a diferença entre "não existe" e "está inativo".

Limitação mantida: `RepositorioBase.estaAtivo(id)` continua devolvendo `false` para inativo **e** para inexistente, porque o `TesteUnitarioManual` do professor usa esse método. A limitação está documentada no Javadoc de `Repositorio.estaAtivo`, que indica `verificarSituacaoCliente/Conta` como a forma correta de distinguir os casos.

Testes utilizados

- Arquivo: `test/java/ddd/SituacaoClienteTest.java` (classe `ddd.SituacaoClienteTest`)
  - `clienteCriadoAtivoDeveEstarNaSituacaoAtivo`
  - `clienteDesativadoDeveEstarNaSituacaoInativo`
  - `clienteReativadoDeveVoltarParaSituacaoAtivo`
  - `deveVerificarSituacaoDeClienteCadastrado`
  - `naoDeveConfundirClienteInexistenteComClienteInativo`
- Também exercitado pelos cenários BDD de ativar/desativar (`ClienteSteps.oClienteDeveraFicarAtivo/Inativo` verificam `getSituacao()`).

---

7 — Remover cliente

Conceito de domínio envolvido

`Cliente` / Repositório de clientes (`GerenciadoraClientes`)

Regra de negócio

Um cliente cadastrado pode ser removido do banco pelo seu id. Remover um id inexistente é rejeitado (`false`). **Não existe no projeto nenhuma restrição** (ex.: exigir cliente inativo ou sem conta), e nenhuma foi inventada.

Classes responsáveis anteriormente

- `RepositorioBase.remove(int id)`, herdado por `GerenciadoraClientes`: devolve `true`/`false`.
- `SistemaBancario` **não** oferecia a operação.

Problema identificado

A regra em si já estava bem representada: remover é uma operação sobre a **coleção** de clientes, portanto é responsabilidade do repositório, não da entidade (um cliente não "remove a si mesmo" do banco). O único problema era que a operação **não estava exposta** no `SistemaBancario`, a fachada por onde passam as outras operações do sistema.

Modelagem proposta

Nova operação de uma linha, que delega ao repositório:

```
public boolean removerCliente(int idCliente) {
    return gerClientes.remove(idCliente);
}
```

Justificativa

Mantém cada responsabilidade no seu lugar: a entidade cuida do próprio estado, o repositório cuida da coleção, e a fachada expõe a operação do caso de uso. Não foi criado nenhum *aggregate* nem serviço extra, porque não havia regra que justificasse.

Observação registrada: como no código original, remover um cliente **não** remove a conta corrente associada (a relação é só o `idContaCorrente` guardado no cliente). O teste `removerClienteNaoRemoveAContaCorrenteAssociada` deixa essa decisão explícita.

Testes utilizados

- Arquivo: `test/java/ddd/RemocaoTest.java` (classe `ddd.RemocaoTest`)
  - `deveRemoverClienteCadastrado`
  - `naoDeveRemoverClienteInexistente`
  - `removerClienteNaoRemoveAContaCorrenteAssociada`
- Também o `TesteUnitarioManual` (remoção via `GerenciadoraClientes`), que continua 25/25.

---

10 — Verificar situação da conta corrente

Conceito de domínio envolvido

`ContaCorrente` / `SituacaoConta`

Regra de negócio

Uma conta corrente está **ATIVA** ou **INATIVA**. Verificar a situação de uma conta inexistente é diferente de ela estar inativa. Não há no projeto regra que dependa da situação da conta (ex.: bloquear transferência), e nenhuma foi criada.

Classes responsáveis anteriormente

- `ContaCorrente`: `boolean ativo`, `isAtivo()`, `setAtivo(boolean)`; `toString()` com `(ativo ? "Ativa" : "Inativa")`.
- `RepositorioBase.estaAtivo(id)`.

Problema identificado

Igual ao do requisito 6: a situação era um `boolean` genérico, e a verificação por id confundia inexistente com inativa.

Modelagem proposta

- Novo enum `negocio.SituacaoConta { ATIVA, INATIVA }`, no feminino, como se fala de conta no banco ("conta ativa").
- `ContaCorrente`: `getSituacao()`, `ativar()`, `desativar()`, `estaAtiva()`; `isAtivo()`/`setAtivo()` mantidos por compatibilidade.
- `SistemaBancario.verificarSituacaoConta(id)` → `Optional<SituacaoConta>`.

Foi criado um enum **separado** de `SituacaoCliente` porque são conceitos diferentes do domínio. Hoje eles têm os mesmos dois valores, mas podem evoluir de forma independente (por exemplo, uma conta "BLOQUEADA").

Justificativa

Mesma do requisito 6: a situação passa a ser um conceito explícito, com nome do negócio e mudanças feitas por ações da própria entidade.

Testes utilizados

- Arquivo: `test/java/ddd/SituacaoContaCorrenteTest.java` (classe `ddd.SituacaoContaCorrenteTest`)
  - `contaCriadaAtivaDeveEstarNaSituacaoAtiva`
  - `contaCriadaInativaDeveEstarNaSituacaoInativa`
  - `contaDesativadaDeveEstarNaSituacaoInativa`
  - `deveVerificarSituacaoDeContaCadastrada`
  - `naoDeveConfundirContaInexistenteComContaInativa`

---

11 — Remover conta corrente

Conceito de domínio envolvido

`ContaCorrente` / Repositório de contas (`GerenciadoraContas`)

Regra de negócio

Uma conta cadastrada pode ser removida pelo seu número (id). Remover um número inexistente é rejeitado. **Não há restrições no projeto** (ex.: impedir remover conta com saldo), e nenhuma foi inventada.

Classes responsáveis anteriormente

- `RepositorioBase.remove(int id)`, herdado por `GerenciadoraContas`.
- `SistemaBancario` não oferecia a operação.

Problema identificado

A implementação existente já representa adequadamente a regra (remoção como operação do repositório). Faltava apenas expô-la no `SistemaBancario`.

Modelagem proposta

```
public boolean removerConta(int idConta) {
    return gerContas.remove(idConta);
}
```

Justificativa

Mesma do requisito 7. Observação: se a conta for removida, um cliente que tenha aquele `idContaCorrente` continua com o número guardado. Como o projeto não define regra para isso, o comportamento foi mantido e documentado.

Testes utilizados

- Arquivo: `test/java/ddd/RemocaoTest.java` (classe `ddd.RemocaoTest`)
  - `deveRemoverContaCorrenteCadastrada`
  - `naoDeveRemoverContaCorrenteInexistente`

Questões do enunciado sobre a situação do cliente

O enunciado pede que o grupo discuta explicitamente algumas perguntas ao analisar o requisito 6 e aplique o mesmo raciocínio à situação e à remoção de contas e clientes. As respostas abaixo registram a posição adotada nesta entrega.

O que significa um cliente estar ativo? No projeto, "ativo" significa que o cliente está na situação `SituacaoCliente.ATIVO`, ou seja, tem relacionamento vigente com o banco. Um cliente inativo continua cadastrado (pode ser consultado, reativado ou removido), mas não é tratado como cliente ativo. Hoje a situação é **informativa**: nenhuma operação do sistema muda de comportamento por causa dela.

Quem deve conhecer essa regra? A própria entidade `Cliente`. É ela que guarda a situação e que muda de estado pelas ações `ativar()` e `desativar()`. O `SistemaBancario` apenas localiza o cliente e pede a ação; o repositório não sabe nada sobre situação (a exceção é o método legado `estaAtivo(id)`, mantido só por compatibilidade).

A situação é apenas um boolean? Não mais. No original era `boolean ativo`, o que tem dois problemas: não comunica o conceito do negócio e não permite crescer. Com o enum, uma situação nova como `BLOQUEADO` ou `ENCERRADO` entra sem trocar o tipo em todo o código, e cada valor já carrega sua descrição ("Ativo", "Inativo"). Pelo mesmo motivo a conta ganhou `SituacaoConta`, separada da do cliente.

Existem operações que não deveriam ser permitidas para clientes inativos? No domínio bancário real, provavelmente sim: por exemplo, abrir uma nova conta ou transferir a partir da conta de um cliente inativo. Porém **o projeto não define nenhuma dessas regras**, e o grupo decidiu não inventá-las, pelos seguintes motivos:

1. O enunciado pede que as alterações sejam justificadas por regras do domínio, e nenhuma fonte do projeto (código, atividade anterior, enunciado) estabelece essa restrição.
2. Hoje a conta não sabe quem é o seu titular: a ligação é apenas o número `idContaCorrente` guardado no cliente. Bloquear a transferência pela situação do cliente exigiria criar essa relação de volta, o que é uma decisão de modelagem maior do que o requisito.
3. Criar uma regra que o professor não pediu mudaria o comportamento testado nos cenários BDD e no teste manual.

Se a regra fosse definida, a modelagem proposta seria: `ContaCorrente.transferir` passaria a exigir `this.estaAtiva()` e `destino.estaAtiva()` dentro de `transferenciaPermitida` (regra da conta), e uma regra envolvendo o **cliente** ficaria no `SistemaBancario`, que é quem enxerga cliente e conta ao mesmo tempo. O ciclo seria o mesmo do TDD: primeiro um teste `naoDeveTransferirDeContaInativa`, depois a implementação.

Essa regra está localizada na classe mais adequada? Depois da modelagem, sim. Antes, a mudança de situação era um `setAtivo(true/false)` chamado de fora, e a verificação por id (`estaAtivo`) ficava no repositório e confundia "inexistente" com "inativo". Agora a mudança de estado está na entidade e a distinção inexistente/inativo está no caso de uso (`verificarSituacaoCliente`, com `Optional`).

O mesmo raciocínio para contas e remoção

- *Situação da conta (req. 10)*: mesmas respostas. `SituacaoConta` é conceito próprio da conta, e nenhuma operação depende dela no projeto atual.
- *Remover cliente (req. 7)*: é possível remover um cliente que ainda tem conta? No projeto, sim, e a conta continua existindo. Isso foi mantido e fixado no teste `removerClienteNaoRemoveAContaCorrenteAssociada` para que a decisão fique visível.
- *Remover conta (req. 11)*: é possível remover uma conta com saldo? No projeto, sim. Um banco real exigiria saldo zero para encerrar a conta; se essa regra fosse definida, ela pertenceria à fachada ou a um serviço de encerramento de conta (que consulta a entidade `possuiSaldo`), e não ao repositório genérico, que não deve conhecer regras específicas de conta.

Onde ficou cada regra (visão geral)

Regra	Classe	Por quê
Idade válida (18–65)	`Cliente.validarIdade`	Invariante da entidade
Ativar/desativar, situação	`Cliente`, `ContaCorrente` + enums	Estado e comportamento da entidade

Regra	Classe	Por quê
Saldo suficiente, débito, crédito, valor positivo, destino válido	`ContaCorrente.transferir` / `transferenciaPermitida`	Regra da conta
Identidade imutável	`Cliente`/`ContaCorrente` (`id` `final`)	Identidade da entidade
Id único no cadastro	`RepositorioBase.adiciona` (lista copiada e somente leitura)	Depende da coleção, não de uma entidade isolada; o repositório é o dono da coleção
Remoção	`Repositorio.remove` (exposto por `SistemaBancario`)	Operação sobre a coleção
Localizar contas para transferir	`GerenciadoraContas.transfereValor`	Coordenação entre duas entidades via repositório